

BestBuy Website Automation Framework

Selenium Pytest & Behave BDD — Project Documentation Report

Project Title: BestBuy E-Commerce Website Automation Framework

Author: Ayush Raj

Employee ID: 4958313

Organization: Wipro Ltd.

Document Type: Capstone Project Documentation

Repository: https://github.com/ayushhraj/AyushRaj_4958313_WiproCapstone

1. Project Description

1.1 Objective of the Project

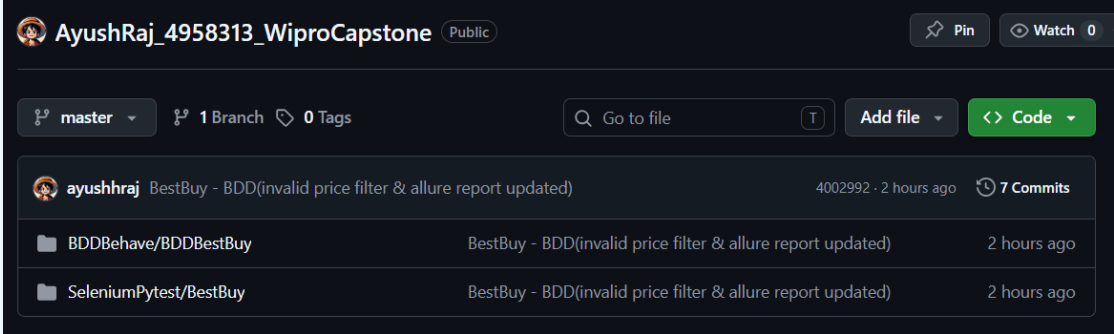
The primary objective of this project is to design and develop a robust, scalable, and maintainable test automation framework for the BestBuy e-commerce website (<https://www.bestbuy.com>). The framework validates critical e-commerce workflows including product navigation, filtering, cart management, and checkout processes through both positive and negative test scenarios.

1.2 Overview of Framework

The project is organized into two independent automation modules:

Module	Framework	Directory
SeleniumPytest	Selenium + Pytest	SeleniumPytest/BestBuy/
BDDBehave	Selenium + Behave BDD	BDDBehave/BDDBestBuy/

Both modules share a consistent architecture based on the Page Object Model (POM) pattern and include identical page classes, utility modules, and test data files. The key difference lies in the test execution approach — Pytest uses Python test classes while Behave uses Gherkin feature files with step definitions.



Screenshot : GitHub Repository — main page showing project name, description, and folder structure

1.3 Purpose of Automation

- Automate repetitive manual test scenarios for the BestBuy website.
- Validate both positive (happy path) and negative (error handling) workflows.

- Ensure the website's critical functionalities work as expected across end-to-end user journeys.
- Generate detailed Allure reports with embedded screenshots and execution logs for traceability.
- Demonstrate professional-grade automation framework architecture for capstone evaluation.

1.4 Features Automated

#	Feature	Type
1	Homepage navigation to Top Deals	Positive
2	TV & Home Theater section navigation	Positive
3	Brand filter application (data-driven from Excel)	Positive
4	Price range filter application (data-driven from Excel)	Positive
5	Adding multiple products to cart	Positive
6	Increasing product quantity in cart	Positive
7	Proceeding to checkout	Positive
8	Complete end-to-end purchase workflow	Positive (E2E)
9	Invalid brand filter validation	Negative
10	Invalid price range filter validation	Negative
11	Invalid email during checkout validation	Negative

1.5 Technologies Used

Technology	Purpose
Python	Programming language
Selenium WebDriver	Browser automation
Pytest	Test framework (SeleniumPytest module)
Behave	BDD framework (BDDBehave module)
Allure	Test reporting (allure-pytest, allure-behave)
OpenPyXL	Reading test data from Excel files
pytest-html	HTML report generation (Pytest module)
Microsoft Edge	Target browser for test execution
webdriver-manager	Automatic WebDriver management

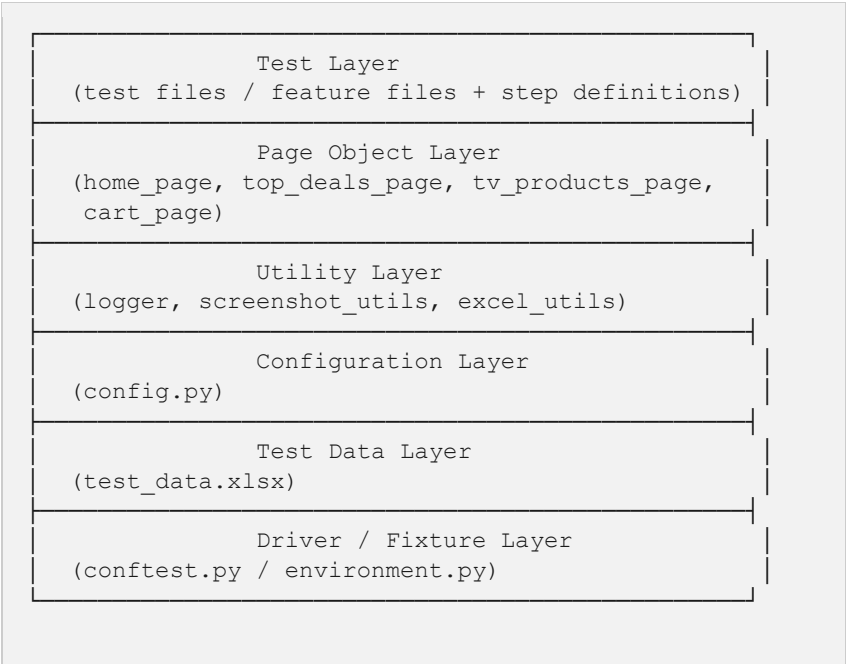
Git / GitHub	Version control and repository hosting
--------------	--

1.6 Framework Architecture

The framework follows a Hybrid Test Automation Architecture combining:

- Page Object Model (POM): Each web page is represented by a dedicated page class containing locators and action methods.
- Data-Driven Testing: Test data is externalized in Excel files and read at runtime using OpenPyXL.
- Utility-Based Design: Reusable utilities for logging, screenshots, and Excel operations are separated into a utilities/ package.
- Configuration Management: Centralized configuration in config/config.py for base URL, wait times, and browser settings.

Layer diagram:



2. Test Strategy

2.1 Type of Framework Implemented

The project implements a Hybrid Automation Framework that combines:

1. Page Object Model (POM) — for modular, maintainable page representations.
2. Data-Driven Testing — for externalizing test data in Excel sheets.
3. Keyword-Driven (via BDD) — Gherkin keywords (Given, When, Then, And) drive test execution in the Behave module.

2.2 Selenium Pytest Implementation

The SeleniumPytest module uses the Pytest testing framework with Selenium WebDriver.

Key Components

- `conftest.py` — Contains the setup fixture that initializes the Edge browser, configures options (maximized window, optional headless mode), navigates to the BestBuy website, selects the "United States" country, and yields the driver instance. After each test the browser is closed via `driver.quit()`. A `pytest_unconfigure` hook generates and opens the Allure report after all tests complete.
- `pytest.ini` — Configures Pytest with `-s -v` flags for verbose output, `--alluredir` for Allure result output, `--html` for HTML report generation, and console logging enabled at INFO level with timestamp formatting.
- `test_bestbuy_workflow.py` — Contains the end-to-end workflow test (`TestBestBuyWorkflow` class) with `@allure.feature` and `@allure.story` decorators.
- `test_positive_negative_tc.py` — Contains the `TestPositiveNegativeTC` class with 5 positive and 3 negative test cases, using `@pytest.mark.parametrize` for data-driven invalid price filter testing.

2.3 BDD Behave Implementation

The BDDBehave module uses the Behave framework for Behavior-Driven Development.

Feature Files (Gherkin syntax)

- `end_to_end.feature` — Defines 1 scenario for the complete TV purchase workflow (8 steps).
- `positive_negative.feature` — Defines 5 positive scenarios and 3 negative scenarios (9 total scenarios).

Step Definitions

- `end_to_end_steps.py` — Implements step definitions using `@given`, `@when`, `@then` decorators, maps Gherkin steps to page object method calls.
- `positive_negative_steps.py` — Implements validation steps and additional step definitions for positive/negative scenarios.

environment.py Hooks

- `before_scenario()` — Launches Edge browser, navigates to BestBuy, selects country, stores driver in `context.driver`.
- `after_scenario()` — Captures screenshot on failure, attaches to Allure, closes browser.
- `after_all()` — Generates and opens Allure report using `subprocess.run()`.

behave.ini Configuration

- Feature paths: `features`
- Allure formatter: `allure_behave.formatter:AllureFormatter`
- Output directory: `reports/allure-results`
- Timing display enabled, skipped tests hidden

2.4 Page Object Model Usage

The Page Object Model is implemented identically in both modules with four page classes:

Page Class	File	Represented Page
HomePage	<code>pages/home_page.py</code>	BestBuy homepage
TopDealsPage	<code>pages/top_deals_page.py</code>	Top Deals section
TVProductsPage	<code>pages/tv_products_page.py</code>	TV & Home Theater product listing
CartPage	<code>pages/cart_page.py</code>	Shopping cart and checkout

Each page class follows a consistent structure:

```
class PageName:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, Config.EXPLICIT_WAIT)
        self.logger = LogGenerator.loggen()

    # Locators defined as class-level tuples
    element_name = (By.XPATH, '//xpath/expression')

    # Action methods that use locators
    def perform_action(self):
```

```

element = self.wait.until(
    EC.element_to_be_clickable(self.element_name)
)
element.click()
ScreenshotUtil.capture_screenshot(self.driver, 'screenshot_name')
self.logger.info('ACTION COMPLETED')

```

Key implementation details:

- Locators stored as class-level tuple variables using (By.XPATH, "...") format.
- WebDriverWait with expected_conditions used for explicit waits throughout.
- JavaScript executor (execute_script) used for click operations where standard clicks may be intercepted.
- scrollView and window.scrollTo used for scrolling to dynamically loaded elements.
- Every action method includes logging, screenshot capturing, and time.sleep() for synchronization.

In the BDDBehave module, page classes additionally include:

- @allure.step() decorators on action methods for step-level Allure reporting.
- allure.attach() calls to attach text-based execution logs directly within page methods.
- A clear_applied_price_filter() method in TVProductsPage for handling multiple invalid price test iterations.

2.5 Data-Driven Testing Implementation

Test data is externalized in an Excel file (test_data/test_data.xlsx) and read at runtime using the ExcelUtils utility class.

Excel Sheets Structure

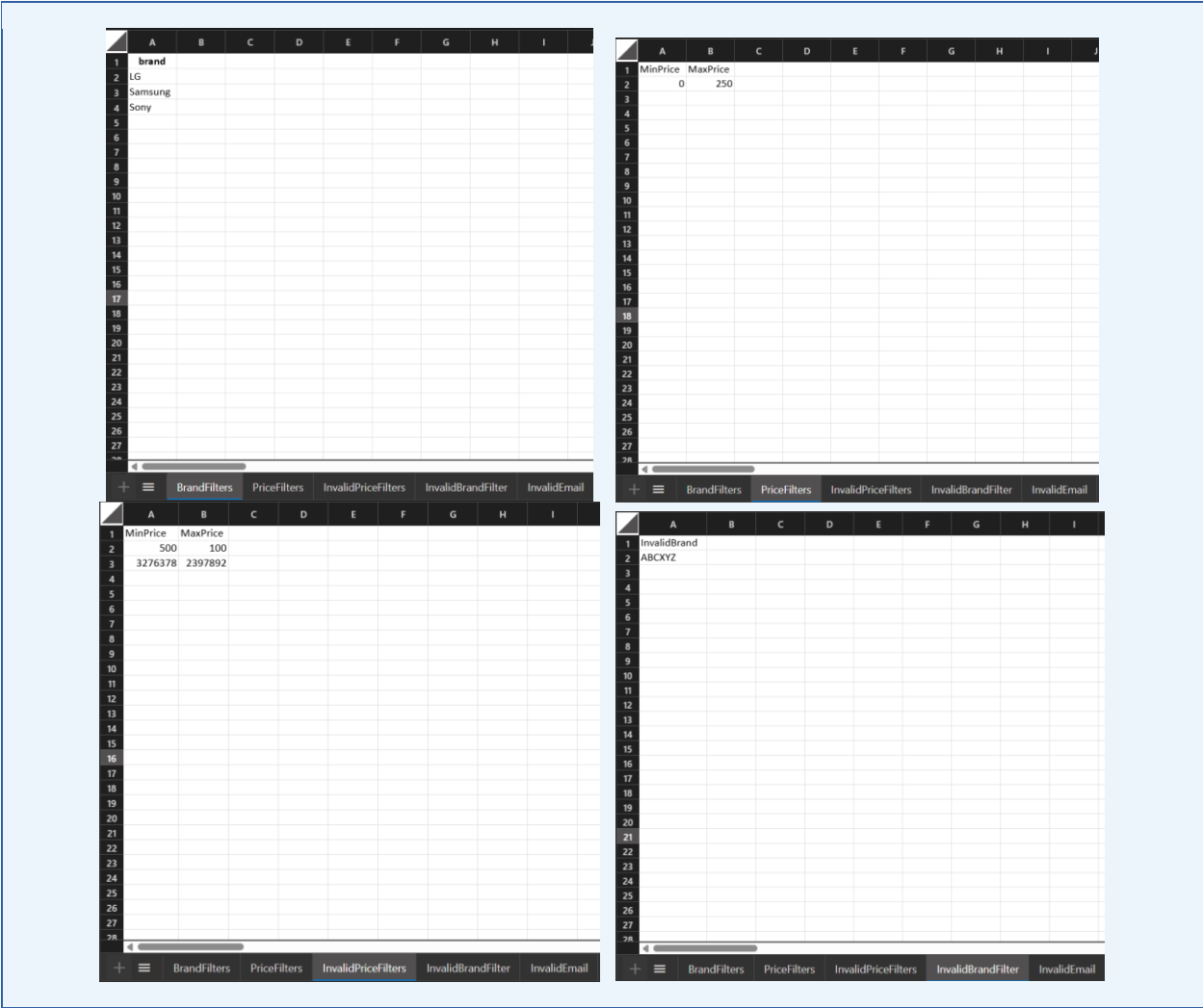
Sheet Name	Columns	Purpose
BrandFilters	Brand Name (Column A)	Valid brand names for filter testing
PriceFilters	Min Price (A), Max Price (B)	Valid price range for filter testing
InvalidPriceFilters	Min Price (A), Max Price (B)	Invalid/reversed price ranges
InvalidBrandFilter	Brand Name (Column A)	Invalid brand name (e.g., ABCXYZ)
InvalidEmail	Email (Column A)	Invalid email for checkout testing

ExcelUtils Methods

Method	Returns	Description
get_brand_data()	list of brand names	Reads all brand names from BrandFilters sheet

get_price_data()	tuple (min, max)	Reads valid price range from PriceFilters sheet
get_invalid_price_data()	list of tuples	Reads all invalid price ranges from InvalidPriceFilters sheet
get_invalid_brand_data()	str	Reads invalid brand name from InvalidBrandFilter sheet
get_invalid_email_data()	str	Reads invalid email from InvalidEmail sheet

In the SeleniumPytest module, @pytest.mark.parametrize is used with Excel data for the invalid price filter test:



	A	B	C	D	E	F	G	H	I
1	InvalidEmail								
2	abc@xyz.com								
3									
4									
5									
6									
7									
8									
9									
10									
11									
12									
13									
14									
15									
16									
17									
18									
19									
20									
21									
22									
23									
24									
25									
26									
27									
28									

Screenshot : Excel Test Data File — test_data.xlsx open showing all 5 sheets

2.6 Logging Implementation

Logging is implemented using Python's built-in logging module via the LogGenerator utility class (utilities/logger.py):

```
class LogGenerator:
    @staticmethod
    def loggen():
        if not os.path.exists('logs'):
            os.makedirs('logs')
        logger = logging.getLogger('BestBuy')
        logger.setLevel(logging.INFO)
        if not logger.handlers:
            file_handler = logging.FileHandler('logs/automation.log')
            formatter = logging.Formatter(
                '%(asctime)s : %(levelname)s : %(message)s'
            )
            file_handler.setFormatter(formatter)
            logger.addHandler(file_handler)
        return logger
```

Key features:

- Logger name: "BestBuy" | Log level: INFO | Log file: logs/automation.log
- Log format: 2026-05-22 16:54:18,123 : INFO : Message
- Duplicate handler prevention — checks if not logger.handlers before adding new handlers.
- Auto-directory creation — creates logs/ folder if it does not exist.
- Used throughout all page classes, test files, step definitions, conftest.py, and environment.py.

pytest.ini also enables live console logging:

```
log_cli = true
log_cli_level = INFO
log_cli_format = %(asctime)s [%(levelname)s] %(message)s
log_cli_date_format = %H:%M:%S
```

2.7 Screenshot Capturing Mechanism

Screenshots are captured using the ScreenshotUtil utility class (utilities/screenshot_utils.py):

```
class ScreenshotUtil:
    @staticmethod
    def capture_screenshot(driver, screenshot_name='screenshot'):
        screenshot_dir = 'screenshots'
        if not os.path.exists(screenshot_dir):
            os.makedirs(screenshot_dir)
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        clean_name = screenshot_name.replace(' ', '_')
        screenshot_path = f'{screenshot_dir}/{clean_name}_{timestamp}.png'
        driver.save_screenshot(screenshot_path)
        allure.attach.file(
            screenshot_path,
            name=clean_name,
            attachment_type=allure.attachment_type.PNG
        )
        logger.info(f'Screenshot captured: {screenshot_path}')
        return screenshot_path
```

Screenshots captured during test execution:

Screenshot Name	Captured At
top_deals	After clicking Top Deals
tv_home_theater	After opening TV & Home Theater section
brand_filters_applied	After applying brand filters
valid_price_filter	After applying valid price filter
invalid_price_filter	After applying invalid price filter
first_product_added	After adding first product to cart
second_product_added	After adding second product to cart
cart_page	After navigating to cart
updated_product_quantity	After updating product quantity
checkout_page	After clicking checkout

invalid_brand	After applying invalid brand filter
invalid_email_entered	After entering invalid email
invalid_email_validation	After clicking continue with invalid email

3. Test Environment

3.1 Environment Details

Component	Details
Operating System	Windows
Browser	Microsoft Edge (Chromium-based)
Programming Language	Python 3.x
IDE	PyCharm
Build / Package Manager	pip
Version Control	Git / GitHub

3.2 Libraries and Dependencies

SeleniumPytest Module — requirements.txt

Library	Purpose
selenium	Browser automation via WebDriver
pytest	Test framework and runner
openpyxl	Reading Excel test data
pytest-html	HTML report generation
webdriver-manager	Automatic WebDriver binary management
allure-pytest	Allure integration for Pytest

BDDBehave Module — requirements.txt

Library	Purpose
selenium	Browser automation via WebDriver
behave	BDD framework
openpyxl	Reading Excel test data
webdriver-manager	Automatic WebDriver binary management
allure-behave	Allure formatter for Behave

allure-python-commons	Common Allure utilities
pytest	Additional test utilities

3.3 Reporting Tools

Tool	Type	Configuration
Allure Framework	Interactive HTML reports	Configured via pytest.ini (--alluredir) and behave.ini (AllureFormatter)
pytest-html	Static HTML report	Configured via pytest.ini (--html=reports/report.html)

3.4 Framework Setup Requirements

To set up and run the framework, the following steps are required:

4. Python 3.x installed on the system.
5. Microsoft Edge browser installed.
6. Allure command-line tool installed and available in system PATH.
7. Install dependencies via pip:

```
# For SeleniumPytest
cd SeleniumPytest/BestBuy
pip install -r requirements.txt

# For BDDBehave
cd BDDBehave/BDDBestBuy
pip install -r requirements.txt
```

4. Test Execution

4.1 Selenium Pytest — Execution Commands

```
# Navigate to project directory
cd SeleniumPytest/BestBuy

# Run all tests
pytest -v -s

# Run specific test file
pytest -v -s tests/test_positive_negative_tc.py

# Run specific test case by name
pytest -v -s tests/test_positive_negative_tc.py -k 'test_increase_product_quantity'
```

4.2 Pytest Execution Flow

```
pytest command triggered
|
▼
pytest.ini loaded (addopts, logging, discovery patterns)
|
▼
conftest.py → setup() fixture invoked
|
├── Edge browser launched (maximized)
├── BestBuy website opened
├── 'United States' country selected
└── WebDriver instance yielded to test
|
▼
Test method executes
|
├── Page objects instantiated (HomePage, TopDealsPage, etc.)
├── Excel data loaded via ExcelUtils
├── Page actions performed (click, filter, add to cart, etc.)
├── Screenshots captured at key steps
├── Assertions validated
└── Logging at each step
|
▼
setup() fixture teardown → driver.quit()
|
▼
pytest_unconfigure() hook
|
├── allure generate reports/allure-results -o reports/allure-report --clean
└── allure open reports/allure-report
```

4.3 Test Classes and Cases — SeleniumPytest

Class: TestBestBuyWorkflow (test_bestbuy_workflow.py)

Test Method	Description
test_complete_workflow	End-to-end workflow: Top Deals → TV Section → Brand Filters → Price Filters → Add Products → Cart → Checkout

Class: TestPositiveNegativeTC (test_positive_negative_tc.py)

#	Test Method	Type	Description
1	test_top_deals_navigation	Positive	Validates Top Deals page navigation
2	test_tv_home_theater_navigation	Positive	Validates TV & Home Theater page navigation
3	test_brand_filters	Positive	Validates brand filter application (Excel-driven)
4	test_price_filters	Positive	Validates price range filter application (Excel-driven)
5	test_increase_product_quantity	Positive	Validates product quantity update in cart
6	test_invalid_price_filter	Negative	Validates invalid price range handling (parameterized with Excel data)
7	test_invalid_brand_filter	Negative	Validates invalid brand search handling
8	test_invalid_email_checkout	Negative	Validates invalid email during checkout

4.4 BDD Behave — Execution Commands

```
# Navigate to project directory
cd BDDBehave/BDDBestBuy

# Run all feature files
behave

# Run a specific feature file
behave features/end_to_end.feature

# Run a specific scenario by name
behave --name '[E2E_TC] Complete TV Purchase Workflow'
```

```
# Run with Allure formatter
behave -f allure_behave.formatter:AllureFormatter -o reports/allure-results
```

4.5 Behave Execution Flow

```

behave command triggered
├─
└─
  behave.ini loaded (paths, formatter, output config)
  └─
    Feature file parsed (Gherkin syntax)
    └─
      environment.py → before_scenario() invoked
      └─
        ┌─ Edge browser launched (maximized)
        ┌─ BestBuy website opened
        ┌─ 'United States' country selected
        └─ driver stored in context.driver
      └─
        Each Gherkin step mapped to step definition function
        └─
          ┌─ @given → step_launch() → initializes page objects on context
          ┌─ @when → step_top_deals(), step_tv_section(), etc.
          └─ @then → assertion step functions
        └─
          environment.py → after_scenario() invoked
          └─
            ┌─ If FAILED: screenshot captured & attached to Allure
            ┌─ If PASSED: logged as 'SCENARIO PASSED'
            └─ driver.quit()
          └─
            environment.py → after_all() invoked (after all scenarios)
            └─
              ┌─ allure generate reports/allure-results -o reports/allure-report --clean
              └─ allure open reports/allure-report

```

4.6 Feature File Scenarios

Feature: end_to_end.feature

```
Feature: BestBuy End To End Workflow
```

```
Scenario: [E2E_TC] Complete TV Purchase Workflow
```

```
Given User launches BestBuy website
```

```
When User opens Top Deals section
```



```

And User opens TV & Home Theater section
And User applies brand filters
And User applies valid price filters
And User adds first two products to cart
And User navigates to cart page
And User proceeds to checkout
Then Checkout page should open successfully

```

Feature: positive_negative.feature (9 scenarios)

- POS_TC_01 — Validate Top Deals Navigation
- POS_TC_02 — Validate TV Navigation
- POS_TC_03 — Validate Brand Filters
- POS_TC_04 — Validate Price Filters
- POS_TC_05 — Validate Product Quantity Update
- NEG_TC_01 — Validate Invalid Price Filter
- NEG_TC_02 — Validate Invalid Brand Filter
- NEG_TC_03 — Validate Invalid Email Checkout

4.7 Gherkin Step-to-Function Mapping

Gherkin Step	File	Python Function
Given User launches BestBuy website	end_to_end_steps.py	step_launch(context)
When User opens Top Deals section	end_to_end_steps.py	step_top_deals(context)
And User opens TV & Home Theater section	end_to_end_steps.py	step_tv_section(context)
And User applies brand filters	end_to_end_steps.py	step_brand_filters(context)
And User applies valid price filters	end_to_end_steps.py	step_price_filters(context)
And User adds first two products to cart	end_to_end_steps.py	step_add_products(context)
And User navigates to cart page	end_to_end_steps.py	step_cart(context)
And User proceeds to checkout	end_to_end_steps.py	step_checkout(context)
Then Checkout page should open	end_to_end_steps.py	step_checkout_validation(context)

And User applies invalid price filters	positive_negative_steps.py	step_invalid_price_filter(context)
Then Invalid price validation should display	positive_negative_steps.py	step_invalid_price_validation(context)
And User applies invalid brand filter	positive_negative_steps.py	step_invalid_brand(context)
Then Invalid brand validation should display	positive_negative_steps.py	step_invalid_brand_validation(context)
And User enters invalid email	positive_negative_steps.py	step_invalid_email(context)
Then Invalid email validation should display	positive_negative_steps.py	step_invalid_email_validation(context)

5. Test Output

5.1 Console Logs

Pytest Console Output

```
===== test session starts =====
collecting ... collected 9 items

tests/test_bestbuy_workflow.py::TestBestBuyWorkflow::test_complete_workflow
16:54:18 [INFO] Opening BestBuy website
16:54:23 [INFO] United States selected
16:54:28 [INFO] SUCCESS : Top Deals opened
...
PASSED

tests/test_positive_negative_tc.py::TestPositiveNegativeTC::test_top_deals_navigation
...
PASSED

=====
TESTS COMPLETED SUCCESSFULLY
GENERATING ALLURE REPORT...
=====
```

Behave Console Output

```
Feature: BestBuy End To End Workflow

  Scenario: [E2E_TC] Complete TV Purchase Workflow
    Given User launches BestBuy website ... passed
    When User opens Top Deals section ... passed
    And User opens TV & Home Theater ... passed
    ...
    Then Checkout page should open successfully ... passed

1 feature passed, 0 failed
9 scenarios passed, 0 failed
```

5.2 Automation Logs

The logs/automation.log file contains detailed timestamped execution logs:

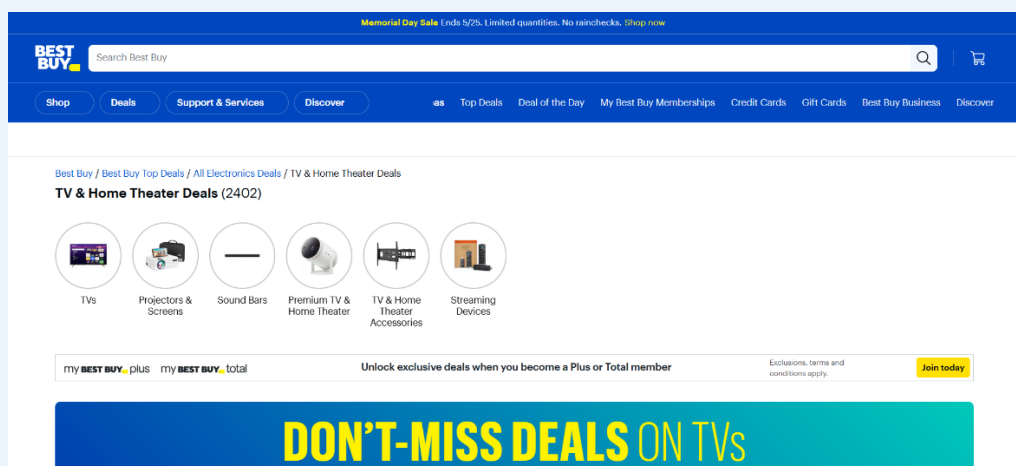
```
2026-05-22 16:54:18,123 : INFO : Opening BestBuy website
2026-05-22 16:54:23,456 : INFO : United States selected
2026-05-22 16:54:28,789 : INFO : SUCCESS : Top Deals opened
```

```
2026-05-22 16:54:28,900 : INFO : Screenshot captured:
screenshots/top_deals_20260522_165418.png
2026-05-22 16:54:33,234 : INFO : SUCCESS : TV & Home Theater opened
2026-05-22 16:54:46,567 : INFO : SUCCESS : Samsung filter applied
2026-05-22 16:54:48,890 : INFO : SUCCESS : LG filter applied
2026-05-22 16:54:50,123 : INFO : SUCCESS : All brand filters applied
...
2026-05-22 17:06:45,456 : INFO : SUCCESS : Invalid email validation completed
```

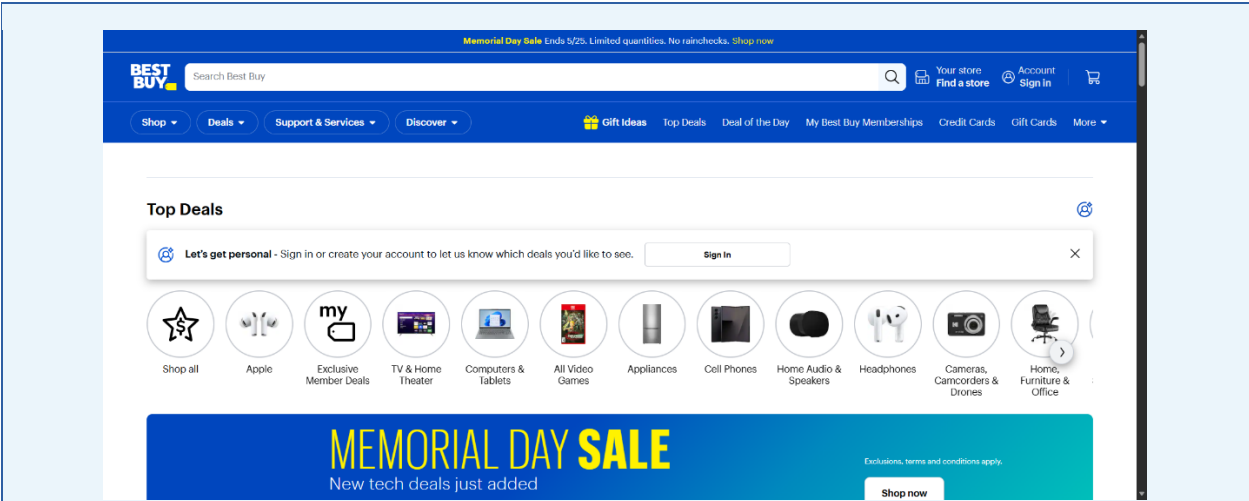
5.3 Generated Screenshots

Screenshots are automatically generated at key test execution points and saved in the screenshots/ directory with timestamp-based naming (format: {name}_{YYYYMMDD_HHMMSS}.png):

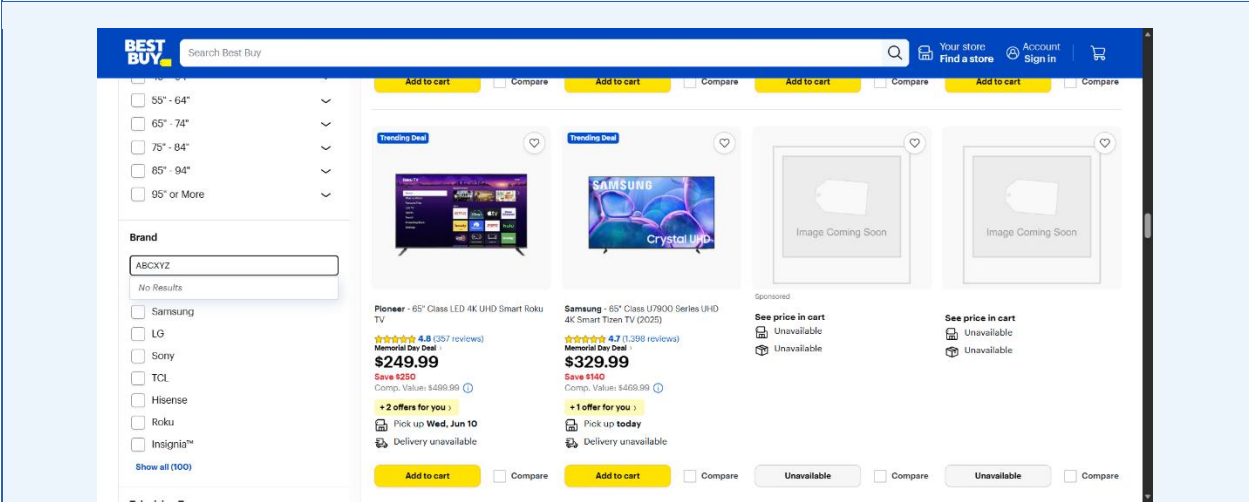
```
screenshots/
├── top_deals_20260522_165418.png
├── tv_home_theater_20260522_165428.png
├── brand_filters_applied_20260522_165446.png
├── valid_price_filter_20260522_165519.png
├── first_product_added_20260522_165522.png
├── second_product_added_20260522_165534.png
├── cart_page_20260522_165543.png
├── checkout_page_20260522_165549.png
├── updated_product_quantity_20260522_170104.png
├── invalid_price_filter_20260522_170215.png
├── invalid_brand_20260522_170418.png
├── invalid_email_entered_20260522_170642.png
└── invalid_email_validation_20260522_170645.png
```



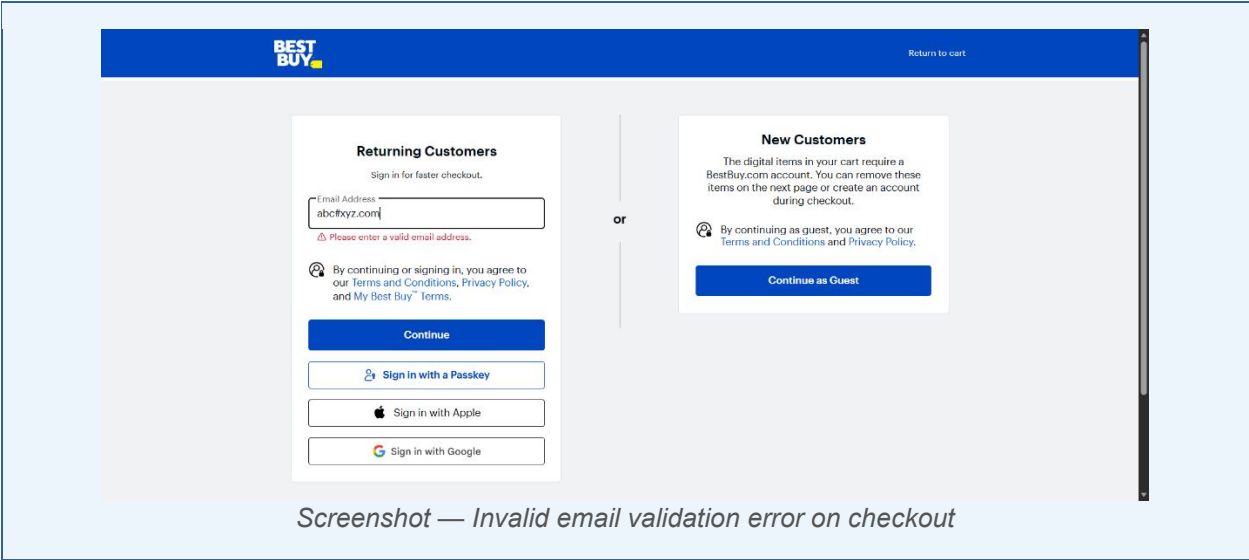
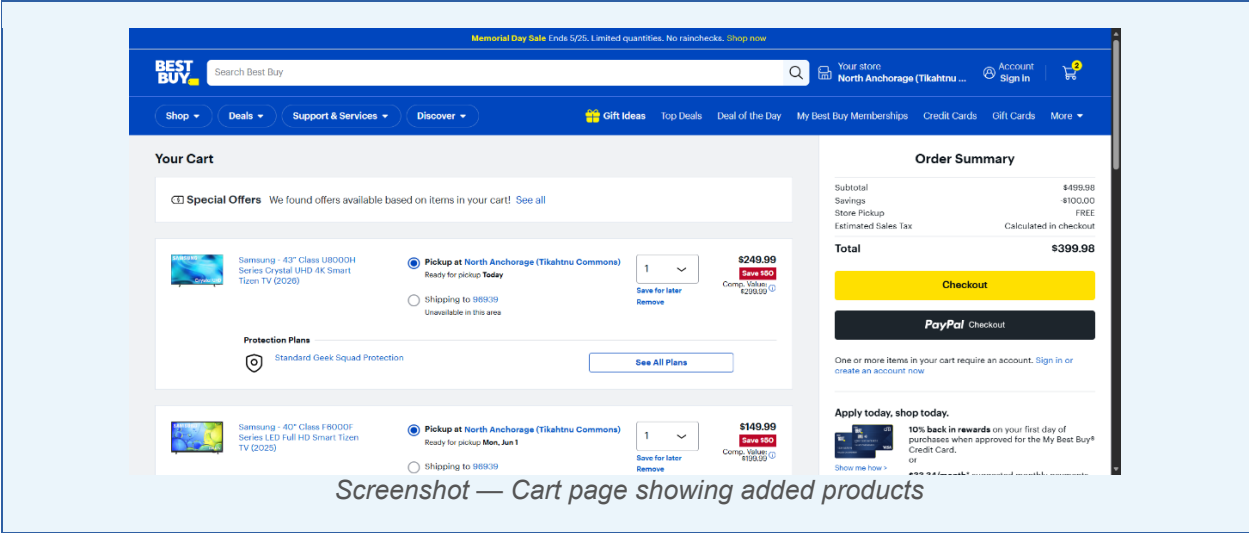
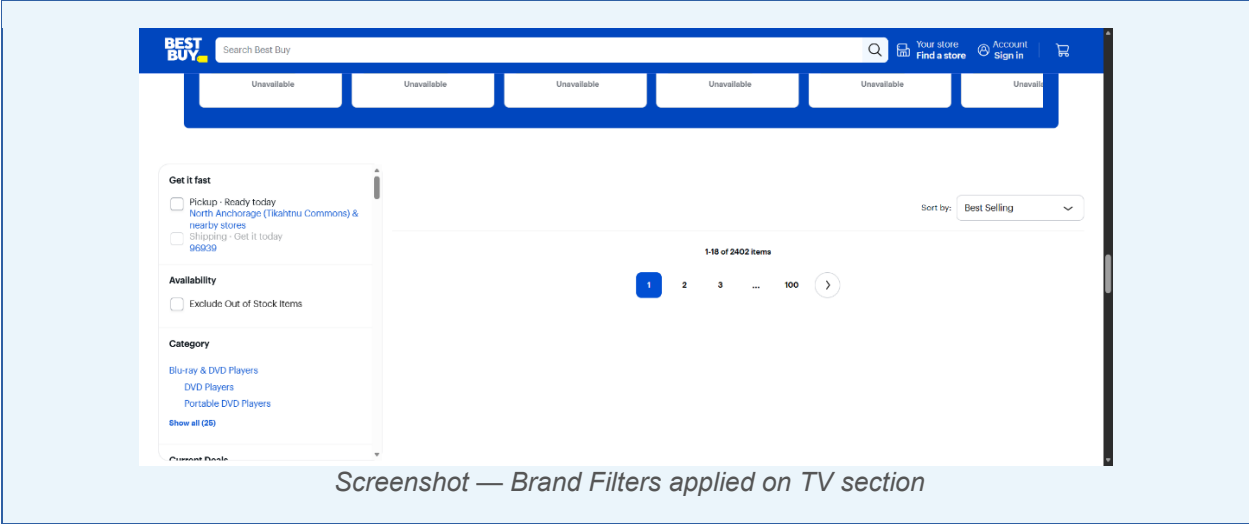
Screenshot — TV & Home Theater page captured during test



Screenshot — Top Deals page captured during test



Screenshot — Invalid brand showing 'No Results' message



5.4 Pass / Fail Validations

Assertion	Test	Purpose
assert "bestbuy.com" in driver.current_url.lower()	E2E Workflow	Validate BestBuy website loaded
assert "top-deals" in driver.current_url	Top Deals Nav	Validate URL contains top-deals
assert "tv" in driver.current_url.lower()	TV Navigation	Validate URL contains tv
assert brand.lower() in driver.page_source.lower()	Brand Filters	Validate brand appears on page
assert min_price < max_price	Price Filters	Validate price range logic
assert "0+to+250" in driver.current_url	Price Filter URL	Validate price filter in URL
assert updated_quantity == "2"	Quantity Update	Validate dropdown selection
assert "No Results" in driver.page_source	Invalid Brand	Validate No Results message
assert "@" not in invalid_email	Invalid Email	Validate email format
assert any(msg in driver.page_source ...)	Invalid Email	Validate error messages on page

Negative test validations:

- Invalid Price Filter: Checks for either "No results found" message OR auto-normalized URL price range.
- Invalid Brand Filter: Verifies "No Results" is displayed in the brand search dropdown.
- Invalid Email: Verifies error messages like "isn't associated with an account", "try a different email address", or "continue as guest".

5.5 Step Execution Output Sample

```
=====
POSITIVE TEST CASE 3 : VALID BRAND FILTER
=====
STEP 1 : Opening Top Deals
SUCCESS : Top Deals opened
STEP 2 : Opening TV Section
SUCCESS : TV & Home Theater opened
STEP 3 : Applying Brand Filters
SUCCESS : Samsung filter applied
SUCCESS : LG filter applied
SUCCESS : All brand filters applied
ASSERTION : Validating Applied Brand Filters
```

```
ASSERTION PASSED : Samsung filter applied successfully
ASSERTION PASSED : LG filter applied successfully
ASSERTION PASSED : Brand Filters Applied Successfully
=====
TEST CASE PASSED
=====
```


6. Test Reports

6.1 Allure Report Implementation

SeleniumPytest Module

- Allure results are generated in reports/allure-results/ directory via the --alluredir option in pytest.ini.
- @allure.feature("BestBuy Ecommerce Workflow") — Feature-level grouping
- @allure.story("Complete TV Purchase Flow") — Story-level grouping
- @allure.title("Complete End-to-End BestBuy Workflow") — Test title
- @allure.description("Validate TV filtering, cart and checkout workflow") — Test description

BDDBehave Module

Allure formatter configured in behave.ini:

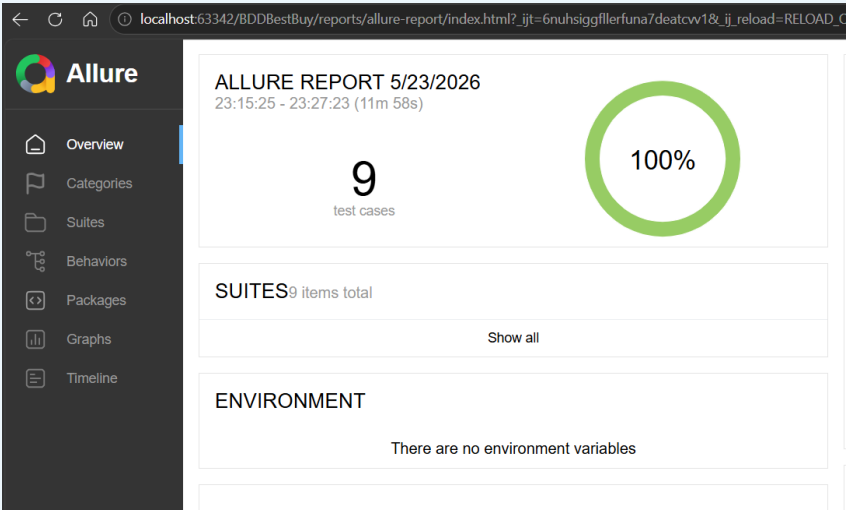
```
format = allure_behave.formatter:AllureFormatter
outfiles = reports/allure-results
```

Additional Allure integrations:

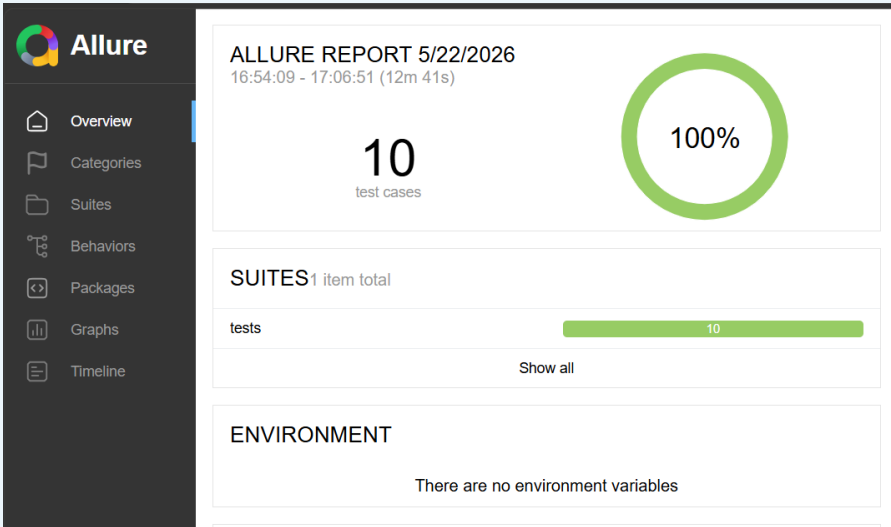
- @allure.step() decorators on page object methods create step-level hierarchy in the report.
- allure.attach() used for text logs, PNG screenshots, and in-memory screenshots.

6.2 Report Sections Available

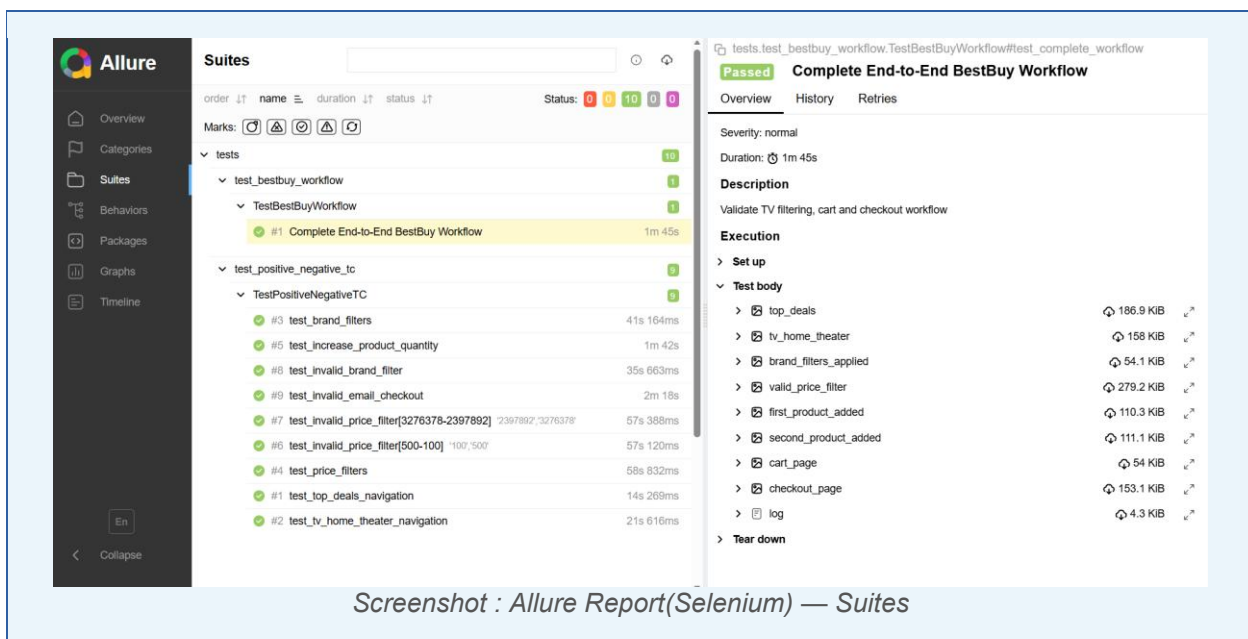
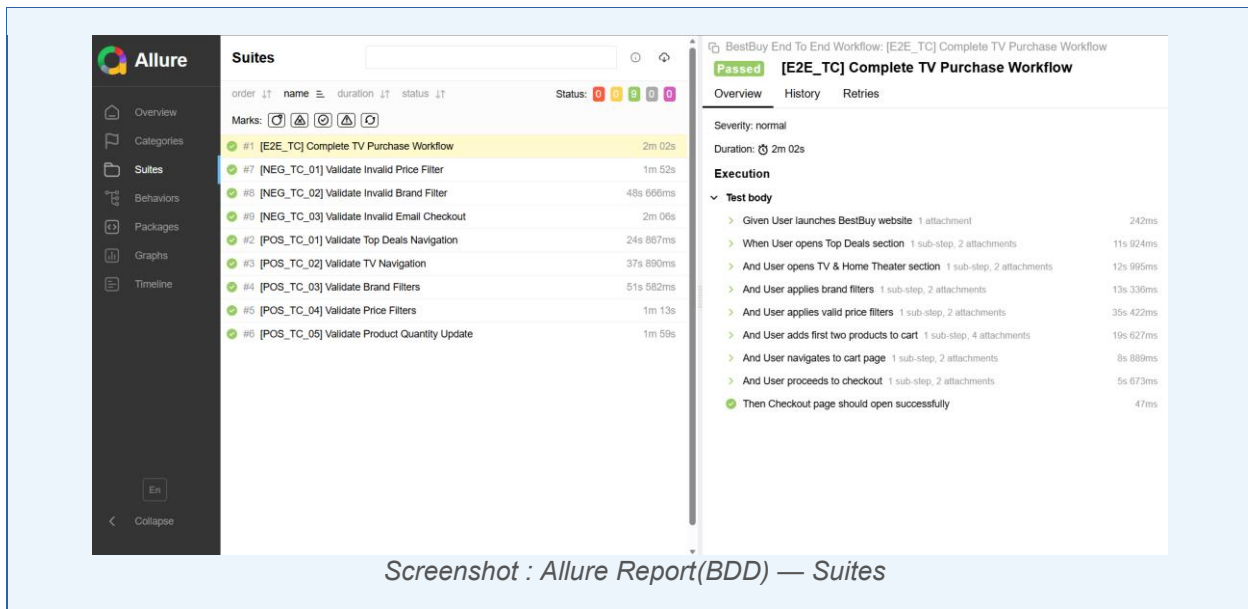
Report Section	Details
Overview Dashboard	Total tests, pass/fail ratio, execution duration, test trends
Suites	Test class hierarchy (TestBestBuyWorkflow, TestPositiveNegativeTC)
Behaviors	Feature → Story → Test Case hierarchy
Timeline	Chronological execution timeline
Categories	Failure categorization
Packages	Package-level grouping



Screenshot : Allure Report(BDD)— Overview Dashboard showing pass/fail statistics



Screenshot : Allure Report(Selenium) — Overview Dashboard showing pass/fail statistics



6.3 Screenshot Attachments in Reports

Screenshots are attached to Allure reports at two levels:

- Step-Level Attachments — via `ScreenshotUtil.capture_screenshot()` which calls `allure.attach.file()` after every screenshot capture.

9. Failure Attachments — via `after_scenario()` in `environment.py` which captures and attaches a screenshot when a scenario fails.

In the Allure report, screenshots appear as expandable image attachments within each test step, allowing evaluators to visually verify the state of the application at each point in the workflow.

6.4 Report Generation Process

SeleniumPytest — Automatic via `pytest_unconfigure()`

```
def pytest_unconfigure(config):
    os.system('allure generate reports/allure-results -o reports/allure-report --clean')
    os.system('allure open reports/allure-report')
```

BDDBehave — Automatic via `after_all()` hook

```
def after_all(context):
    subprocess.run(
        'allure generate reports/allure-results -o reports/allure-report --clean',
        shell=True, check=True
    )
    subprocess.run(
        'allure open reports/allure-report',
        shell=True, check=True
    )
```

Generated Report Files

```
reports/
├─ allure-results/      # Raw Allure result JSON files
├─ allure-report/       # Generated Allure HTML report
└─ report.html          # Pytest HTML report (SeleniumPytest only)
```

7. Project Folder Structure

7.1 SeleniumPytest Module

```
AyushRaj_4958313_WiproCapstone/
├── SeleniumPytest/
│   └── BestBuy/
│       ├── config/
│       │   ├── __init__.py
│       │   └── config.py                # Base URL, waits, headless mode
│       ├── pages/
│       │   ├── __init__.py
│       │   ├── home_page.py            # HomePage class (Top Deals)
│       │   ├── top_deals_page.py       # TopDealsPage class (TV section)
│       │   ├── tv_products_page.py     # TVProductsPage class (filters, cart)
│       │   └── cart_page.py            # CartPage class (checkout, email)
│       ├── tests/
│       │   ├── __init__.py
│       │   ├── test_bestbuy_workflow.py # E2E workflow test
│       │   └── test_positive_negative_tc.py # 5 positive + 3 negative tests
│       ├── utilities/
│       │   ├── __init__.py
│       │   ├── logger.py                # LogGenerator class
│       │   ├── screenshot_utils.py      # ScreenshotUtil class
│       │   └── excel_utils.py           # ExcelUtils class
│       ├── test_data/
│       │   └── test_data.xlsx           # Excel test data (5 sheets)
│       ├── screenshots/                 # Auto-generated screenshots
│       ├── logs/
│       │   └── automation.log            # Execution logs
│       ├── reports/
│       │   ├── allure-results/
│       │   ├── allure-report/
│       │   └── report.html
│       ├── conftest.py                  # Pytest fixtures & hooks
│       ├── pytest.ini                   # Pytest configuration
│       └── requirements.txt              # Dependencies
```

7.2 BDDBehave Module

```
AyushRaj_4958313_WiproCapstone/
├── BDDBehave/
│   └── BDDBestBuy/
```

```
├── config/
│   ├── __init__.py
│   └── config.py                # Base URL, waits, headless mode
├── features/
│   ├── end_to_end.feature      # E2E scenario (Gherkin)
│   ├── positive_negative.feature # 5 positive + 3 negative scenarios
│   ├── environment.py         # Behave hooks (before/after)
│   └── steps/
│       ├── __init__.py
│       ├── end_to_end_steps.py # E2E step definitions
│       └── positive_negative_steps.py
├── pages/
│   ├── __init__.py
│   ├── home_page.py           # HomePage class (with @allure.step)
│   ├── top_deals_page.py
│   ├── tv_products_page.py
│   └── cart_page.py
├── utilities/
│   ├── __init__.py
│   ├── logger.py
│   ├── screenshot_utils.py
│   └── excel_utils.py
├── test_data/
│   └── test_data.xlsx
├── screenshots/
├── logs/
│   └── automation.log
├── reports/
│   ├── allure-results/
│   └── allure-report/
├── behave.ini                  # Behave configuration
└── requirements.txt            # Dependencies
```

— End of Documentation —
